

CSCE 629 ANALYSIS OF ALGORITHMS · MODULE 09 · LECTURE NOTES

Greedy Algorithms and Exchange Arguments

When taking the best option now is provably right, and how to tell.

MODULE OUTCOMES

- Identify the greedy choice property and optimal substructure.
- Prove greedy correctness by exchange argument.
- Prove greedy correctness by 'greedy stays ahead'.
- Recognise standard problems where greed fails, and why.
- Construct counterexamples to refute a proposed greedy rule.

1 What greedy means

A greedy algorithm builds a solution incrementally, at each step taking whatever looks best right now and never reconsidering. It is the simplest possible strategy, it is usually fast, and it is usually wrong — which is exactly why the proof matters more here than almost anywhere else in the course.

1.1 The two requirements

- **Greedy choice property.** There is an optimal solution containing the greedy first choice. This is the non-trivial condition and the one the proof techniques of §2 establish.
- **Optimal substructure.** After committing to the greedy choice, what remains is an instance of the same problem, and an optimal solution to the whole contains an optimal solution to the remainder.

Optimal substructure is shared with dynamic programming. The difference is what you do with it: dynamic programming considers every choice at each step and keeps the best; greedy commits to one and never looks back. Greedy is therefore dramatically faster when the greedy choice property holds, and silently incorrect when it does not.

THE DISCIPLINE THIS MODULE IS REALLY ABOUT

Plausible greedy rules are easy to invent and usually wrong, and their failure mode is unkind: they work on every small example you try by hand, then fail on a structured input months later.

So treat an unproved greedy algorithm as a **conjecture**. Either produce an exchange argument or produce a counterexample. 'It seems to work on my tests' is not one of the available states.

2 Proof technique 1: exchange arguments

The structure is always the same three steps:

- 1 Let OPT be an arbitrary optimal solution.
- 2 Find the first place where OPT disagrees with the greedy solution, and exchange OPT's choice there for greedy's. Argue the result is still feasible and no worse.
- 3 Conclude by induction that the greedy solution is itself optimal.

The cut property of Module 08 was an exchange argument: given an MST omitting the lightest edge across a cut, we swapped that edge in and a heavier one out, and showed the result was still a spanning tree of no greater weight.

2.1 Worked example: interval scheduling

Given intervals with start and finish times, select as many mutually non-overlapping intervals as possible.

Rule: repeatedly take the compatible interval that finishes earliest.

Proof. Let greedy produce $g_1, g_2, \dots$ and let OPT be an optimal solution $o_1, o_2, \dots$, both sorted by finish time. Suppose they first differ at index k . Greedy chose g_k as the earliest-finishing interval compatible with $g_1 \dots g_{k-1}$, which equal $o_1 \dots o_{k-1}$. Since o_k was also compatible with that prefix, greedy had it available, so $\text{finish}(g_k) \leq \text{finish}(o_k)$.

Replace o_k with g_k in OPT. It is compatible with the prefix by construction, and since it finishes no later than o_k , it cannot conflict with o_{k+1} or anything after. The modified solution has the same size, so it is still optimal, and it agrees with greedy one step further. Induction completes the argument. ■

WHY EARLIEST FINISH AND NOT SOMETHING ELSE

Finishing earliest leaves the maximum possible room for everything that follows. That is the entire intuition, and it is what the exchange step formalises.

Other plausible rules fail, and quickly: **shortest interval first** fails on one short interval that overlaps two long non-overlapping ones. **Earliest start first** fails on a single long interval that starts first and blocks everything. **Fewest conflicts first** fails too, though the counterexample is larger.

Three plausible rules, all wrong. This is the normal situation.

3 Proof technique 2: greedy stays ahead

Define a measure of partial progress, then show that after each step greedy's measure is at least as good as that of any other solution after the same number of steps. If greedy leads throughout, its final answer cannot be worse.

For interval scheduling the measure is the finish time of the k -th selected interval: greedy's k -th always finishes no later than any other solution's k -th, so greedy can never run out of room first. Same conclusion as the exchange argument by a different route.

Which technique to use is largely taste. Exchange arguments are more general; stays-ahead is often more natural when the problem has an obvious running quantity — elapsed time, remaining capacity, coverage so far.

4 Where greed fails

Problem	Plausible rule	Why it fails
0/1 knapsack	Highest value-to-weight ratio first	The best-ratio item can occupy space that two slightly worse items would have used to greater total value. NP-hard.
Shortest path with negative edges	Closest unfinished vertex first	Dijkstra's exchange argument needs prefix $\leq$ whole path, which negative edges break (Module 07).
Set cover	Set covering the most uncovered elements	Greedy is not optimal, but it is an $H_n \approx \ln n$ approximation — and that is provably the best possible unless $P = NP$ (Module 13).
Coin change	Largest coin that fits	Correct for US and Euro denominations, wrong in general. With coins $\{1, 3, 4\}$, greedy makes 6 as $4+1+1$ (three coins) instead of $3+3$ (two).
Graph colouring	Colour vertices greedily in some order	Highly order-dependent. For some graphs and orderings greedy uses arbitrarily more colours than necessary.

THE COIN CHANGE CASE IS THE CAUTIONARY ONE

Greedy coin change is optimal for the US and Euro systems, which are the ones everyone tests with. The algorithm therefore looks correct to anyone who does not try a pathological denomination set.

This is precisely the failure mode described in §1: works on every case you naturally try, wrong in general. The general problem needs dynamic programming (Module 10).

4.1 Fractional versus 0/1 knapsack

Fractional knapsack: items may be divided. Sort by value/weight ratio and fill the sack greedily, taking a fraction of the last item. This is optimal, by a straightforward exchange argument — if an optimal solution contains any of a lower-ratio item while a higher-ratio item remains available, swap a small amount and the value strictly increases.

0/1 knapsack: items are indivisible. The same rule fails, and the problem is NP-hard (Module 12).

The structural difference is that the exchange step requires being able to swap an arbitrarily small amount. With indivisible items that move is unavailable, and the exchange argument — and with it the greedy choice property — collapses. One word in the problem statement moves it from polynomial to NP-hard, which is worth remembering the next time someone describes two problems as 'basically the same'.

5 A method for attacking greedy rules

- 1 **Try to break it first.** Spend ten minutes looking for a counterexample before spending an hour on a proof. Probe small cases, extreme values, ties, and items that are nearly indistinguishable. Counterexamples are cheap and proofs are expensive, so search in that order.

- 2 **If it survives, attempt an exchange argument.** Ask where greedy and a hypothetical optimum could first disagree, and whether you can swap without loss.
- 3 **If the exchange fails, look at why.** The step that will not go through almost always points directly at the counterexample.
- 4 **If greed genuinely fails, change technique.** Dynamic programming if the problem has overlapping subproblems (Module 10); approximation if it is NP-hard (Module 13).

6 The greedy algorithms worth knowing

Problem	Rule	Proof technique
Interval scheduling	Earliest finish time	Exchange
Interval partitioning	Sort by start; assign to any free resource	Stays ahead (the answer equals the maximum overlap depth)
Fractional knapsack	Highest value/weight ratio	Exchange
Huffman coding	Repeatedly merge the two least frequent symbols	Exchange
Minimum spanning tree	Lightest edge that is safe	Cut property (Module 08)
Dijkstra	Closest unfinished vertex	Exchange (Module 07)
Job sequencing with deadlines	Highest profit first, scheduled as late as possible	Exchange / matroid

Note that Dijkstra and the MST algorithms appear here. They were greedy algorithms when you met them; this module supplies the vocabulary and the proof technique that were implicit at the time.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
MIT 6.046J — Greedy Algorithms	Interval scheduling and the exchange argument, done carefully.
Jeff Erickson, Algorithms — Greedy Algorithms chapter	Unusually good on how to <i>find</i> the right greedy rule and on the discipline of trying to break it first. Includes several instructive wrong rules.
Tim Roughgarden — Greedy Algorithms (Part 3)	Huffman coding and scheduling, with the proofs spelled out at length.
CSES Problem Set — Sorting and Searching section	Many of these are greedy problems in disguise. Good practice at recognition.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement interval scheduling with four rules: earliest finish, earliest start, shortest duration, fewest conflicts. Find a counterexample for each of the three incorrect ones.
- 2 Write out the exchange argument for fractional knapsack in full, then construct a 0/1 instance where the same rule is suboptimal and explain which step of the proof fails.
- 3 Implement greedy coin change. Verify it is optimal for US denominations, then find three coin systems where it is not. Write a brute-force checker to search for such systems automatically.
- 4 Implement Huffman coding and prove the greedy choice property: the two least frequent symbols are siblings at maximum depth in some optimal tree.
- 5 Implement interval partitioning (assign lectures to the fewest rooms). Prove by a stays-ahead argument that the number of rooms used equals the maximum number of overlapping intervals.
- 6 Take a greedy rule you believe is correct for some problem of your own, and spend ten minutes trying to break it before attempting a proof. Report which you found.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What two properties must hold for a greedy algorithm to be correct, and which one is the hard one?
- 2 What distinguishes greedy from dynamic programming, given that both need optimal substructure?
- 3 Describe the three steps of an exchange argument.
- 4 Why is 'earliest finish time' the correct rule for interval scheduling, and why does 'shortest interval' fail?
- 5 Give the structural reason fractional knapsack is greedy-solvable while 0/1 knapsack is NP-hard.
- 6 Why should you try to break a greedy rule before trying to prove it?